

VOLUME III

Security, Telemetry, Evaluation and Operations

A detailed control manual for trust boundaries, guardrails, observability, model risk, testing, SLOs, deployment, backup, incident response, scaling, governance, and production readiness.

FOCUS

What the platform records, how it fails safely, how operators know it is healthy, how reviewers approve it, and what must change before institutional scale.

Implementation snapshot: 02 September 2026

Implemented behavior is separated from planned production extensions on every page.

Contents and reading guide

Read after Volumes I and II. This volume explains how the platform protects data and decisions, what telemetry is implemented, what evaluation proves, how quarterly operations run, and which production controls are still planned.

Status convention

IMPLEMENTED means executable behavior in this repository. PLANNED means a designed production extension that is catalogued or declared but not wired. BOUNDARY means a deliberate non-goal or limitation.

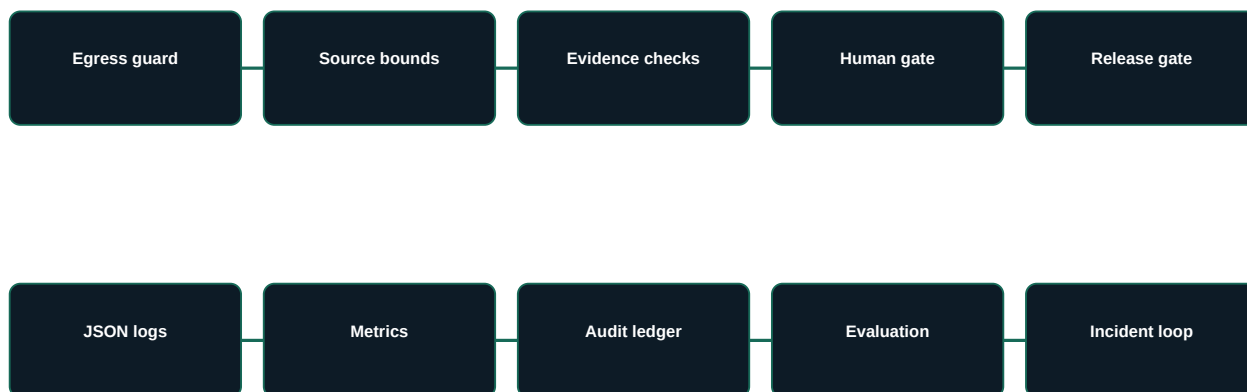
Contents and reading guide	2
Control planes at a glance	5
Trust model	6
Threat model	7
Prompt-injection guardrail	8
SSRF and network-egress guardrail	9
Credential and secret guardrail	10
Connector-authority guardrail	11
Transport guardrails	12
Evidence-integrity guardrails	13
Point-in-time and look-ahead guardrails	14
Financial-authority guardrail	15
Scoring guardrails	16
Human-control guardrails	17
Operational-isolation guardrail	18
API authentication boundary	19
Data privacy, retention, and model processing	20

Telemetry architecture	21
Structured JSON logging	22
Timed operations	23
Process metrics captured	24
Prometheus endpoint and limitations	25
Persistent audit events	26
Correlation and trace model	27
Observability signal matrix	28
Alerts and operating thresholds	29
Service-level objectives	30
OpenTelemetry target design	31
LLM telemetry and model-risk capture	32
Evaluation philosophy	33
Implemented run-evaluation metrics	34
Research-ready and publication-ready gates	35
Score reproducibility evaluation	36
Golden benchmark design	37
Agent-specific evaluation	38
Red-team corpus	39
Executable test suite	40
Change control and model governance	41
Governance roles and separation of duties	42
Local and single-node deployment	43
Docker and Compose deployment	44

Kubernetes quarterly CronJob	45
Quarterly operating runbook	46
Backup strategy	47
Restore and reproducibility drill	48
Incident response	49
Failure-semantics matrix	50
Scaling path	51
PostgreSQL migration	52
S3 or MinIO migration	53
Temporal outer orchestration	54
Redis, analytics, and vector retrieval	55
Run-to-run comparison	56
Report and manifest reproducibility	57
Current workspace implementation snapshot	58
Known limitations	59
Recommended roadmap	60
Production readiness acceptance criteria	61
Operational checklist	62
Volume III references	63

Control planes at a glance

Security and observability are cross-cutting planes. They surround the evidence and research pipeline rather than appearing only at the API edge.



Operating invariant

When the platform cannot prove identity, evidence authority, numeric validity, or approval, it preserves the record and withholds the rank or publication.

SECTION 01

Trust model

IMPLEMENTED

The system treats the internet, parser output, model output, and analyst override as separate trust boundaries.

Internet content is untrusted and may be malicious, inaccurate, stale, or contractually restricted. Parser output is untrusted until identity, date, unit, schema, and lineage checks pass. Model output is untrusted until exact-quote, type, range, unit, period, confidence, contradiction, and human-review checks pass. Analyst decisions are privileged and must be authenticated and audited.

Published reports contain only approved claims and deterministic calculations. Trust increases through explicit verification stages; it is never granted because a source looks professional or a model sounds confident.

Concrete contract

```
UNTRUSTED INTERNET -> HASHED RAW -> VALIDATED DOCUMENT -> PROPOSAL  
-> HUMAN-ACCEPTED CLAIM -> DETERMINISTIC ASSESSMENT -> APPROVED REPORT
```

SECTION 02

Threat model

IMPLEMENTED

Threats include malicious content, unsafe networking, evidence manipulation, model drift, secret leakage, and privileged misuse.

Threat	Primary control
Indirect prompt injection	Quoted document context, signature flags, role prompt, no document tool authority
SSRF	HTTPS only, public DNS/IP, redirect revalidation
Hallucinated figures	Exact quote, schema, unit/range/period checks, human acceptance
Data poisoning	Source tiers, hashes, contrary evidence, review
Look-ahead bias	Frozen cutoff and filing-date filters
Secret leakage	Environment-only credentials and no secret-bearing state
Cross-company leakage	Company-scoped state and retrieval
Silent model/prompt drift	Versioned configuration and benchmark diff
Ranking manipulation	Deterministic code and reproducibility gate
Malicious override	Production RBAC + comments + audit
Supply-chain compromise	Pinned ranges, SBOM, signatures, scans

SECTION 03

Prompt-injection guardrail

IMPLEMENTED

Retrieved instructions are data, never authority.

The security module scans text for phrases such as ignore previous instructions, system prompt, developer message, reveal prompt/secret, call this tool, and exfiltration. Filing normalization scans up to the first 100,000 characters; normalized evidence packages scan document previews; discovery titles are scanned. Matches are stored as injection flags and counted.

Detection is not the only defense. The common and extraction prompts explicitly state that source text is untrusted quoted material. Model contexts expose no retrieval, database-write, publication, or external-action tools. Exact quotes must still match the supplied chunk. A flagged document can be reviewed or excluded without executing any embedded instruction.

Why this design

Signature scanning catches common attacks but can miss obfuscation. The decisive controls are least privilege, quoted context, deterministic validation, and human acceptance.

SECTION 04

SSRF and network-egress guardrail

BOUNDARY

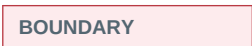
External data access is restricted to public HTTPS destinations and every redirect is revalidated.

`validate_external_url` rejects non-HTTPS schemes, missing hosts, localhost, .local names, private addresses, loopback, link-local, and reserved networks. Source requests resolve DNS and check every resolved address. HTTPX automatic redirects are disabled so up to six redirects can be followed manually with the same validation.

A production cluster should add network-policy or proxy enforcement because application validation alone cannot defend against DNS rebinding, compromised dependencies, or alternate egress paths. Source workers should reach only approved domains and the model gateway; scoring, ranking, and reporting need no internet access.

SECTION 05

Credential and secret guardrail

A diagram consisting of a red rectangular box with a thin black border. Inside the box, the word "BOUNDARY" is written in a bold, black, sans-serif font, centered horizontally and vertically.

Secrets are referenced by environment variable name and injected only at the transport boundary.

Settings loads .env without overriding a value already supplied by the process. Source definitions contain only an auth type, header/query name, environment-variable name, and optional flag. SourceHttpClient reads the value immediately before the request. Public source metadata may reveal the required variable name and whether it is configured, but never the value.

The model API key and pilot API key follow the same environment-only rule. Secrets do not enter source manifests, workflow state, prompts, report manifests, or audit payloads. Production should use short-lived credentials from a secret manager, distinct development/production accounts, spend and rate limits, and immediate rotation after disclosure.

SECTION 06

Connector-authority guardrail

IMPLEMENTED

Configuration can select only adapters already registered in code.

SourceIngestionService owns a connector dictionary containing sec_edgar, gleif, and gdelt. sync fails when a source is disabled, planned, missing a credential, or names an unregistered connector. The catalog cannot supply an import path, code expression, shell command, or arbitrary tool.

This fail-closed registry is a supply-chain boundary. Adding a provider requires a reviewed code change plus fixtures and governance, not only a configuration edit.

SECTION 07

Transport guardrails

IMPLEMENTED

Per-source rate, time, retry, redirect, header, and body limits bound connector behavior.

Control	Implemented behavior
Rate	Monotonic minimum interval from catalog requests/second
Timeout	HTTPX timeout per source
Retries	1-8 attempts; 429/5xx and network errors only
Backoff	Retry-After clamped 0.25-30 sec; exponential fallback capped
Redirects	Maximum six, validated at every hop
Response bytes	Per-source hard cap after retrieval
Headers	Small safe allow list
Errors	Sanitized type/status without credentials

SECTION 08

Evidence-integrity guardrails

IMPLEMENTED

Every accepted claim carries enough structure to validate and reproduce its meaning.

Required fields include company, document, claim type, numeric/text value, unit, period, confidence, exact evidence span, page/section or chunk, source tier, publication date, cutoff eligibility, contradiction flag, and extraction method. Database foreign keys connect claims to the company and document.

Proposal validation checks exact contiguous quote inclusion, permitted claim type, numeric finiteness/range, exact unit, narrative/numeric exclusivity, ISO period, and confidence cap. The skeptic checks required-claim coverage and Tier-1 authority for financials. The system retains invalid/rejected proposals and contradictory claims rather than hiding them.

SECTION 09

Point-in-time and look-ahead guardrails

IMPLEMENTED

The research clock is enforced at request, source selection, document selection, claim query, and run-manifest levels.

A future run date is rejected. SEC filing and XBRL facts must be filed on or before the cutoff. Evidence extraction filters documents by publication date. Database list_claims excludes later and as_of_eligible=false records. The run stores the cutoff and selected company IDs.

Historical corrections never replace the published record in place. A restatement or taxonomy change triggers a new run. Production should extend effective dating to security identity, currency rates, project states, and provider revisions.

SECTION 10

Financial-authority guardrail

IMPLEMENTED

Reported revenue and operating margin require Tier-1 evidence and deterministic period alignment.

The margin agent requests a non-contradictory Tier-1 claim. The skeptic independently checks Tier-1 total revenue and operating margin. The SEC parser filters annual forms and fiscal periods, selects the most recent cutoff-eligible fact, and requires the operating-income period to equal the revenue period.

This prevents a Tier-3 article, model narrative, quarterly/annual mismatch, or stale insertion from supplying a critical margin input. Adjusted and segment measures may be stored separately but cannot silently replace the documented rubric.

SECTION 11

Scoring guardrails

IMPLEMENTED

Deterministic functions reject missing, non-finite, out-of-range, and unordered inputs before ranking.

Implementation details

- AI exposure must be within [0,1].
- Segment CAGR must be finite and greater than -100%.
- Forecast years must be positive.
- Every weighted component must exist and be within [0,5].
- Weight sum must be positive.
- Maximum risk discount must be within [0,1].
- Bear <= base <= bull.
- Moat and risk discount remain within ranges.
- Margin must be finite.
- Ties use company ID for deterministic order.

SECTION 12

Human-control guardrails

BOUNDARY

Evidence acceptance and report publication require explicit, persisted human decisions.

Only pending, validation-clean proposals can be accepted. The reviewer and comment are stored on the proposal and in an audit event. Company reviews capture reviewer, decision, comment, and optional override metadata. The reference implementation records overrides but does not provide a separate deterministic override recalculation engine.

The publication gate permits only completed/published runs and verifies that every ranked company is approved. Production must associate these actions with SSO/RBAC identities and separate research, methodology, data, and operations permissions.

SECTION 13

Operational-isolation guardrail

IMPLEMENTED

A bad company, source, document, or model response should fail within its smallest safe boundary.

Source failures close their sync attempt, preserve `cursor_before`, record a sanitized error, and leave existing evidence intact. Invalid model proposals remain invalid and cannot be accepted. Expected dossier insufficiency becomes `validation_errors` rather than an exception. Unexpected company exceptions create a non-rankable assessment while other futures continue.

The supervisor fails the entire run only when a portfolio-level operation cannot complete safely. Publication is always fail-closed. The incident process creates a corrected run instead of rewriting historical rows.

SECTION 14

API authentication boundary

BOUNDARY

The reference X-API-Key is a pilot control, not an institutional identity system.

All /api/v1 operations depend on `require_api_key`, which compares the request header with `AIFACTORY_API_KEY`. The workbench HTML, /health, /ready, and /metrics are unprotected. Input schemas bound common abuse cases, but there is no per-user role, tenant, session, object authorization, or rate limit.

Before shared deployment, place the service behind SSO/OIDC, implement RBAC and per-action audit identity, protect the UI session, restrict metrics/readiness exposure, add API and provider quotas, and retrieve secrets from a managed vault.

SECTION 15

Data privacy, retention, and model processing

PLANNED

The intended data is public-company research, but licensed text, analyst notes, and provider logs still require policy.

Do not send confidential analyst notes, personal data, or licensed full transcripts to an unapproved free model endpoint. Retain only what the provider licence permits. Logs should contain identifiers, counts, error types, and evidence references rather than full documents or hidden reasoning.

Production retention classes should cover raw public evidence, licensed documents, proposals, accepted claims, audit events, reports, security logs, and backups separately. Deletion must not break the integrity of a published report; where rights require deletion, retain a tombstone and impact record.

SECTION 16

Telemetry architecture

BOUNDARY

The reference implementation combines structured JSON logs, process-local Prometheus metrics, timed operations, and persistent SQLite audit events.

Logs answer what happened now, metrics answer how often and how long in the current process, and audit events answer who changed research state across restarts. Context variables attach `run_id` and `company_id` to logs without passing them through every function call.

The OTEL endpoint and optional OpenTelemetry packages are declared but no exporter is wired. The current metrics registry is in-memory and has no labels, histograms, persistence, or multi-process aggregation. Those limitations are explicit rather than implied by dependency declarations.

SECTION 17

Structured JSON logging

IMPLEMENTED

Every application log line is machine-readable and correlation-aware.

Field	Source and meaning
timestamp	UTC ISO timestamp generated by formatter
level	Python logging severity
logger	Module/logger name
message	Sanitized human-readable event
run_id	Context variable for portfolio run
company_id	Context variable for dossier
exception	Formatted traceback when present
event	Optional operation/event name
agent	Optional graph node
duration_ms	Optional timed duration
status	ok or error

Concrete contract

```
{"timestamp": "...Z", "level": "INFO", "logger": "aifactory.service",  
"message": "research_run completed", "run_id": "...", "company_id": "",  
"event": "research_run", "duration_ms": 1250.4, "status": "ok"}
```

SECTION 18

Timed operations

IMPLEMENTED

A context manager records duration and error count for source fetches, research runs, and every agent invocation.

`timed_operation` captures `perf_counter` at entry. On exception it sets `status=error`, increments `{operation}_errors_total`, and re-raises. In finally it records a duration observation and emits a structured log. Agent completion uses debug level; other operations use info.

Current duration summaries expose only count and sum. Production should export histogram buckets or exemplars with `run/source/agent` attributes, while avoiding company ticker as a high-cardinality metric label when a trace/log correlation ID is sufficient.

Concrete contract

```
with timed_operation('source_fetch', logger): ...  
with timed_operation('research_run', logger): ...  
with timed_operation('agent_execution', logger, agent=name): ...
```

SECTION 19

Process metrics captured

IMPLEMENTED

The registry exposes counters and summary count/sum series with the aifactory_ prefix.

Metric family	Meaning
source_fetch_total	Successful bounded source responses
source_fetch_retry_total	Transport/provider retries
source_fetch_errors_total	Timed source operation errors
source_sync_completed_total	Completed/not-modified source syncs
source_sync_failed_total	Failed source syncs
source_prompt_injection_flags_total	Documents with detected signatures
agent_{name}_total	Executions per graph node
agent_execution_errors_total	Unhandled node errors
company_workflow_failures_total	Isolated company exceptions
research_runs_completed_total	Successful portfolio runs
research_runs_failed_total	Failed portfolio runs
*_seconds_count/sum	Timed operation summary

SECTION 20

Prometheus endpoint and limitations

BOUNDARY

GET /metrics renders the in-memory registry as Prometheus text.

Metric names replace non-alphanumeric characters with underscores and add the aifactory_ prefix. Counters emit a TYPE line and value. Duration summaries emit TYPE summary plus seconds_count and seconds_sum. A lock protects concurrent updates.

The endpoint is process-local. Restarting resets metrics; multiple workers do not aggregate; labels, buckets, HELP text, and exemplars are absent. Production should use a Prometheus client or OpenTelemetry SDK and a collector, and should protect the endpoint from unauthenticated external access.

SECTION 21

Persistent audit events

IMPLEMENTED

Audit events preserve research and governance actions after logs and process metrics are gone.

audit_events stores UUID, optional run and company IDs, actor, event_type, JSON payload, and creation time. The list endpoint supports run-scoped review.

Event	Actor	Key payload
source_sync_completed/failed	source_ingestion_service	source, scope, status, counters or error type
evidence_extraction_completed	evidence_proposal_service	documents, pending/invalid, policy, model
claim_proposal_reviewed	reviewer	proposal, decision, claim ID, comment
run_started/completed/failed	workflow_supervisor/ranking_service	counts, runtime, error type
company_assessed	workflow_supervisor	rankable, score, validation errors
assessment_reviewed	reviewer	decision, comment, overrides
run_published	actor	report and manifest paths
demo_seeded / package_ingested	system/service	record counts

SECTION 22

Correlation and trace model

PLANNED

`run_id` and `company_id` are the primary correlations; source sync and proposal IDs add operation-level lineage.

The company workflow sets context variables before executing nodes and resets them in finally. The service sets run context for the portfolio. Source syncs, proposals, documents, claims, assessments, and reviews each have stable IDs that appear in database relationships and audit payloads.

Production OpenTelemetry should create one quarterly-run trace, child source-sync and company-workflow spans, and node/model/parser spans. Attributes should include versions, source ID, agent name, status, counts, and hashes - not raw prompts, secret values, licensed text, or hidden reasoning.

SECTION 23

Observability signal matrix

PLANNED

A complete operating view connects sources, parsers, models, agents, scoring, review, and publication.

Stage	Signals to capture	Primary diagnosis
Source	availability, HTTP status, latency, retries, freshness, bytes	Provider or network failure
Parser	raw/normalized counts, duration, failure reason, hash change	Format drift and data quality
Entity	method, score, ambiguity, override	Wrong issuer/security
Model	provider/model, latency, tokens/cost, schema error, proposal count	Drift, budget, invalid output
Agent	duration, validation errors, evidence coverage	Research bottleneck
Score	reproducibility, sensitivity, non-finite rejection	Method/code regression
Review	queue age, acceptance/rejection reasons, approval coverage	Human bottleneck
Publication	gate failures, report hash, delivery	Governance/release issue

SECTION 24

Alerts and operating thresholds

PLANNED

Alerts should correspond to user impact or research-integrity risk, not every warning.

Implementation details

- Any failed scheduled quarterly run.
- More than 5% company-workflow failures.
- Source freshness beyond a segment/provider threshold.
- Repeated provider 429/5xx or credential failures.
- Citation coverage below 85% or declining materially.
- Score reproducibility below 100%.
- Unexpected Top-20 churn without material evidence change.
- Proposal invalid/rejection rate outside baseline.
- Model latency, token use, or cost above budget.
- Publication attempted with incomplete approval.
- Backup or restore drill failure.

SECTION 25

Service-level objectives

PLANNED

Initial SLOs balance research integrity, scheduled reliability, read availability, and recoverability.

SLO	Initial target	Measurement source
Scheduled run completion	99%	research_runs + scheduler
Score reproducibility	100%	evaluate_run
Required numeric citation coverage	100%	claim/assessment audit
Mean full-dossier citation coverage	>=95%	evaluation metrics
Rankable workflow success	>=95%	assessment/ranking counts
API read availability	99.5%	external probe
Restore point objective	24 hours	backup policy
Restore time objective	4 hours	restore drills

SECTION 26

OpenTelemetry target design

PLANNED

The production target exports traces and metrics through an organization collector without leaking research content.

The pyproject already declares OpenTelemetry API, SDK, and OTLP HTTP exporter extras, and Settings includes AIFACTORY_OTEL_ENDPOINT. Wiring should create resource attributes for service/version/environment, instrument FastAPI and HTTPX, and wrap source sync, parser, extraction, company graph, agent, scorer, report, and publication operations.

Use sampling and redaction policies. Do not export full prompts, model completions, raw documents, API keys, authorization headers, analyst comments containing sensitive data, or hidden reasoning. Store stable hashes and IDs so authorized operators can join telemetry to the evidence ledger.

SECTION 27

LLM telemetry and model-risk capture

PLANNED

Model observability must measure behavior without turning telemetry into a second ungoverned data lake.

Capture provider, model, endpoint class, prompt/extraction-policy version, request duration, completion cap, token counts when returned, cost estimate, HTTP error class, JSON parse outcome, required-key outcome, proposal count, invalid reasons, human acceptance/rejection, and fallback usage.

Do not capture credentials, full licensed content, private analyst notes, or hidden reasoning. Sample prompt/completion bodies only in an approved evaluation store with explicit retention and access. Hash canonical request context to detect repeatability without retaining everything in logs.

SECTION 28

Evaluation philosophy

IMPLEMENTED

The platform evaluates deterministic correctness, evidence fitness, scenario stability, and human readiness separately.

A single LLM-as-judge score would conceal whether a failure came from identity, source quality, arithmetic, unsupported claims, or writing style. `evaluate_run` therefore recomputes numeric outputs from stored fields and reports multiple metrics and release gates.

Model/agent quality needs a separate point-in-time analyst-labelled benchmark. LLM judges may supplement human labels only after calibration and must preserve judge prompt/model versions. Production model changes cannot affect default proposals until the benchmark and rank-diff gates pass.

SECTION 29

Implemented run-evaluation metrics

IMPLEMENTED

evaluate_run reads the persisted run, assessments, and rankings and derives eight integrity metrics.

Metric	Calculation / interpretation
assessment_count	All persisted company dossiers
ranked_count	Portfolio outputs after Top-N
rankable_rate	rankable assessments / all assessments
mean_evidence_confidence	Mean tier-adjusted claim confidence by dossier
mean_citation_coverage	Mean required-claim coverage
score_reproducibility_rate	Stored score matches deterministic recomputation within 1e-9
unsupported_required_claim_count	Validation errors containing missing/lack
mean_scenario_rank_stability	Bear/bull spread normalized across ranking
ranked_approval_coverage	Approved ranked companies / ranked companies

SECTION 30

Research-ready and publication-ready gates

IMPLEMENTED

Evaluation separates a methodologically usable result from one authorized for external release.

Research-ready requires 100% score reproducibility, mean citation coverage at or above policy, mean evidence confidence at or above policy, and nonempty rank output. Publication-ready requires all research gates plus 100% ranked approval coverage.

These are portfolio-level checks. A company can still be excluded for its own missing evidence while the remaining portfolio is research-ready, provided the rankable-rate and operating policies are acceptable. Production should add minimum universe coverage and segment-balance gates.

SECTION 31

Score reproducibility evaluation

IMPLEMENTED

Every stored risk-adjusted score is recalculated from the stored moat, margin score, company CAGR, and risk discount.

The evaluator multiplies `moat_score`, `operating_margin_score`, `forecast.base_company_ai_cagr`, 100, and one minus `risk_discount`. Absolute difference from stored `risk_adjusted_tafgs` must be at most $1e-9$. The rate must equal one for the release gate.

A failure indicates code/version drift, corrupt persistence, unexpected override behavior, or a report built from inconsistent data. The appropriate response is to pause publication, identify the affected run and version, and create a corrected run after remediation.

SECTION 32

Golden benchmark design

PLANNED

Before production model changes, create a frozen analyst-labelled set of at least 30-50 companies across all five capital layers.

Implementation details

- Preserve the exact source bundle and cutoff analysts saw.
- Include exact XBRL and margin labels.
- Cover currency, unit, period, and accounting-basis normalization.
- Include public-parent, ADR, dual-listing, merger, and ambiguous-name cases.
- Label exposure intervals and methods.
- Label all moat and risk components with rationale evidence.
- Define bear/base/bull drivers and walk-forward outcomes.
- Include unsupported, contradictory, and prompt-injection documents.
- Measure rank sensitivity and quarterly churn.
- Test narrative no-new-facts and citation correctness.

SECTION 33

Agent-specific evaluation

PLANNED

Each role has a metric aligned with its own decision boundary.

Agent	Primary evaluation
Eligibility	Identity precision/recall and ambiguous-match escalation
Evidence	Tier/coverage calculation and contradiction recall
Exposure	Analyst interval overlap and materiality error
Moat	Component-level agreement and evidence coverage
Margin	Exact numeric and accounting-period match
Growth	Driver coverage, scenario ordering, walk-forward calibration
Risk	Category recall and severity agreement
Skeptic	Unsupported-claim recall versus false rejection
Narrative	No-new-facts rate and citation correctness

SECTION 34

Red-team corpus

IMPLEMENTED

Adversarial documents test the full evidence boundary, not only prompt wording.

Implementation details

- Ignore-previous-instructions and fake tool-call content.
- Hidden HTML/PDF text and white-on-white instructions.
- Conflicting currencies and unit multipliers.
- Later-dated restatements inserted into historical runs.
- Subsidiary names designed to map to the wrong public parent.
- Duplicate filings with different hashes.
- News estimates presented as reported company facts.
- Malformed JSON and schema-confusing output.
- Oversized responses, redirect chains, and unsafe DNS targets.
- Analyst override attempts without correct role.

Why this design

The expected outcome is a flagged, invalid, rejected, or non-rankable record. The system never takes autonomous corrective action against an external source.

SECTION 35

Executable test suite

IMPLEMENTED

The repository contains 29 tests across configuration, scoring, security, ingestion, extraction, sources, model routing, and end-to-end publication.

Suite	Representative checks
test_config	Dotenv does not override runtime values
test_security	Injection flag, HTTPS/private-network rejection
test_scoring	Margin boundaries, finiteness, exposure materiality, risk, deterministic score
test_ingestion	XBRL cutoff/period and prompt flag persistence
test_llm	OpenRouter URL, JSON parsing, schema and token cap
test_sources	Secret redaction, idempotent storage, cursors, auth, response cap
test_extraction	Model proposal requires review before evidence
test_end_to_end	Seed -> run -> rank -> report -> review -> publish; missing evidence excludes

SECTION 36

Change control and model governance

PLANNED

Any source tier, parser, prompt, model, taxonomy, formula, risk weight, or threshold change is a research-method release.

Implementation details

- Assign a new version.
- Run the frozen benchmark.
- Produce extraction, score, and rank differences.
- Measure segment and subgroup regressions.
- Review invalid/rejection and abstention changes.
- Confirm data-rights and security impact.
- Obtain methodology and model-risk approval.
- Deploy with rollback and retain the prior version.

SECTION 37

Governance roles and separation of duties

PLANNED

Method, data, research, operations, model risk, and security require distinct accountability.

Role	Owns	Must not do silently
Research owner	Methodology, taxonomy, thresholds	Change defaults without benchmark
Data steward	Sources, rights, retention	Enable unreviewed provider
Analyst	Evidence/dossier review and release	Rewrite historical score
Platform operator	Reliability, deployment, restore	Alter scoring data
Model-risk reviewer	Model/prompt approval	Approve based only on demos
Security owner	Threat model, egress, incident	Expose secrets or weaken controls

SECTION 38

Local and single-node deployment

IMPLEMENTED

The simplest supported deployment runs one process with a persistent artifact volume.

A Python environment installs the base package and launches Uvicorn. Settings point to config/, artifacts/aifactory.db, artifacts/raw-sources, and artifacts/reports. One API/workflow process avoids SQLite multi-writer conflicts. The operator supplies a non-default API key and a real SEC contact User-Agent.

This mode is appropriate for a developer or controlled pilot. Backups must capture database, raw artifacts, reports, and config versions together. The workbench and metrics exposure should be restricted if the host is shared.

SECTION 39

Docker and Compose deployment

BOUNDARY

The image uses Python 3.12 slim, installs the package, runs as non-root UID 10001, exposes port 8000, and includes a health check.

compose.yaml fails closed unless AIFACTORY_API_KEY and AIFACTORY_SEC_USER_AGENT are supplied. It mounts one named volume at /app/artifacts and maps source/model environment variables. The container starts Uvicorn on 0.0.0.0 and restarts unless stopped.

For production, build in CI, pin the base image digest, generate an SBOM, scan dependencies and image layers, sign the image, use a private registry, inject secrets at runtime, and restrict egress.

SECTION 40

Kubernetes quarterly CronJob

BOUNDARY

The supplied manifest schedules a prohibited-overlap refresh at 06:00 UTC on the fifth day of each quarter-opening month.

The placeholder image must be replaced. More than one writer requires PostgreSQL first. Plain Kubernetes Secret objects should be replaced by a secret-manager integration in an institutional deployment.

Field	Configured value
schedule	0 6 5 1,4,7,10 *
concurrencyPolicy	Forbid
successful history	4
failed history	4
backoffLimit	2
restartPolicy	Never
command	aifactory quarterly-refresh --generate-report
secret	aifactory-secrets
volume	aifactory-data PVC at /app/artifacts

SECTION 41

Quarterly operating runbook

IMPLEMENTED

The scheduled command starts automation; the release remains a controlled research process.

Implementation details

- Confirm source licences, credentials, and connector health.
- Freeze the research cutoff.
- Sync filings, IR, project, technical, and discovery evidence.
- Review taxonomy and capital-weight assumptions.
- Validate parser outputs and adjudicate proposals.
- Execute the company graph and portfolio rank.
- Evaluate the run and compare with the previous quarter.
- Resolve contradictions and investigate unexpected churn.
- Review every provisional Top-20 dossier.
- Publish and archive the report, manifest, benchmark, and approvals.

SECTION 42

Backup strategy

PLANNED

A valid backup is a consistent set of transactional data, raw evidence, reports, and method versions.

On the reference node, stop writes or use SQLite's online backup API, then copy the database, raw-sources directory, reports directory, and exact configuration release together. Hash the backup inventory and test restoration quarterly.

Production uses PostgreSQL point-in-time recovery, versioned object retention, immutable report manifests in a separate location, encryption, access logging, and cross-region copies according to policy. Redis caches and in-memory metrics are not authoritative backup data.

SECTION 43

Restore and reproducibility drill

PLANNED

A restore is complete only when the system can reproduce the stored research output, not merely open the database.

Implementation details

- Restore the database to the selected timestamp.
- Restore raw and report object versions.
- Verify content hashes and missing-object inventory.
- Load the exact configuration/prompt release.
- Recompute stored assessment scores.
- Regenerate the report and match its SHA-256.
- Confirm audit chronology and review state.
- Measure RPO/RTO and document gaps.

SECTION 44

Incident response

PLANNED

Integrity incidents pause publication while preserving evidence acquisition and forensic history when safe.

Implementation details

- Pause publication and scheduled scoring.
- Identify affected runs, sources, documents, prompts, models, parsers, and credentials from manifests.
- Preserve raw evidence, logs, audit events, and provider response metadata.
- Revoke compromised credentials or disable the connector/model route.
- Assess rank/report impact and notify owners.
- Correct code, source, or policy under change control.
- Create a new run; do not rewrite historical rankings.
- Record cause, remediation, validation, and communication.

SECTION 45

Failure-semantics matrix

IMPLEMENTED

Every layer has a defined safe failure state and retained evidence.

Boundary	Behavior
Source retrieval	Bounded retry; failed sync; cursor unchanged; sanitized error
Parser	Raw bytes preserved; parser failure recorded; alternate parser may retry
Model	Invalid/error proposal operation; no accepted claim
Agent validation	Error accumulated; dossier non-rankable
Company exception	Persist isolated failure; portfolio continues
Scoring	Reject non-finite/missing/range/order errors
Portfolio	Run fails only if fan-in/persistence cannot complete safely
Publication	Conflict until every ranked dossier is approved
Correction	New immutable run and rank diff

SECTION 46

Scaling path

PLANNED

Scale only after measuring real source, parser, model, and company-workflow load.

Implementation details

- Keep one process and SQLite while validating method and evidence quality.
- Move repositories to PostgreSQL before multiple writers.
- Move raw and rendered documents to versioned object storage.
- Separate API replicas from workers.
- Use Temporal for durable quarterly/source workflows.
- Partition company workflows and bound each provider's concurrency.
- Use Redis for distributed rate limits and locks, not evidence.
- Cache by document hash plus prompt/policy version, never ticker alone.
- Add OTLP, SSO/RBAC, secret manager, and egress policy.
- Load-test, restore-test, and red-team before broad access.

SECTION 47

PostgreSQL migration

PLANNED

The production database must implement the same repository semantics with migrations and multi-writer transactions.

The pyproject declares psycpg, SQLAlchemy, and Alembic, but installing them does not migrate storage. Implement a repository interface, translate schema and indexes, add effective-dated identity where required, use JSONB judiciously, and preserve uniqueness/idempotency keys.

Migration should backfill SQLite data with counts and hashes, run dual-read or shadow verification, compare all assessments and ranks, then cut over writers. Connection pools, transaction isolation, statement timeouts, row-level access, PITR, and migration rollback become operational responsibilities.

SECTION 48

S3 or MinIO migration

PLANNED

RawSnapshotStore is the seam for moving immutable evidence out of the local filesystem.

Use keys that preserve source ID, retrieval date, hash prefix, digest, and extension. Store sidecar metadata or object tags for source URL, content type, external ID, parser version, rights, and retention. Enable versioning, encryption, checksums, object lock where appropriate, and lifecycle rules.

The database should store bucket/key/version ID plus SHA-256 rather than a local path. Retrieval for parsing must verify the digest. Reports and rendered PDF pages require separate prefixes and retention classes.

SECTION 49

Temporal outer orchestration

PLANNED

Temporal is planned for durable source, quarterly, and portfolio workflows; LangGraph remains the bounded company research graph.

Temporal workflows can schedule source syncs, wait on rate-limited activities, retry transient provider failures, fan out company child workflows, record heartbeats, survive process restarts, and enforce idempotency. Human evidence and publication approvals can appear as signals or external state checks.

Activities must use stable keys and avoid repeating irreversible side effects. The workflow history should store IDs and statuses, not licensed source bodies. LangGraph can execute inside a company activity because its role is local typed reasoning, not infrastructure durability.

SECTION 50

Redis, analytics, and vector retrieval

PLANNED

Declared production extras support coordination and analysis but do not replace the evidence ledger.

Component	Permitted role	Not authoritative for
Redis	Rate limits, locks, short caches	Evidence or review decisions
DuckDB/Polars	Batch QA, benchmark analysis, rank diff	Transactional system of record
pgvector	Semantic candidate retrieval	Exact citation or accepted claim
Arele	Taxonomy-aware XBRL validation	Analyst exposure estimate
Playwright	Approved JS-only acquisition	Scoring worker browser authority

SECTION 51

Run-to-run comparison

IMPLEMENTED

`compare_runs` explains portfolio movement without mutating either run.

Rankings are keyed by company. A company in both runs is retained with `rank_change` = previous rank minus current rank and `score_change` = current minus previous. A new company is entered; a missing company is exited. Results sort by current rank with exits last.

The comparison should be joined with evidence, policy, parser, and model diffs to explain causality. Production reporting should classify movement as source update, identity change, claim review, methodology change, scenario change, or operational exclusion.

SECTION 52

Report and manifest reproducibility

IMPLEMENTED

The report is a presentation of approved database records; the manifest is the verification handle.

ReportGenerator reads the run and ranking rows, renders methodology and company profiles, writes Markdown, and calculates its SHA-256. The adjacent JSON manifest stores run ID, cutoff, status, generation time, taxonomy/scoring/prompt/model versions, ranked count, report hash, and sorted unique evidence claim IDs.

An auditor can reload the run, resolve assessments and claims, regenerate the report, and compare the hash. Publication writes a run_published audit event naming both artifacts.

SECTION 53

Current workspace implementation snapshot

BOUNDARY

The local artifact database demonstrates the platform lifecycle but should not be confused with a production research publication.

The repository includes a 20-company synthetic fixture explicitly marked demo and live connector artifacts. Counts are a September 2026 workspace snapshot, not a claim of complete global coverage or an investment recommendation.

Record type	Current count
Companies	25
Market segments	5
Source documents	67
Evidence claims	402
Entity identifiers	11
Source cursors	4
Source sync runs	9
Claim proposals	8
Research runs	6
Company assessments	43
Rankings	43
Audit events	66

SECTION 54

Known limitations

BOUNDARY

The reference platform is real and testable, but several institutional capabilities remain deliberately incomplete.

Implementation details

- Valuation and expected return are absent.
- Global regulator coverage is incomplete.
- PDF/page-aware parsing is not implemented.
- SQLite/local files support one controlled node, not horizontal writers.
- API authentication is a single header key.
- Metrics are local and OTLP export is unwired.
- Outer orchestration is not durable across process loss.
- Review overrides are recorded but not deterministically recalculated.
- Security master effective dating and corporate actions need expansion.
- Licensed data rights and vendor contracts require procurement review.

SECTION 55

Recommended roadmap

PLANNED

Implementation order follows research value and control maturity rather than adding the largest number of connectors.

Order	Capability	Reason
1	Harden SEC + GLEIF pilot	Reliable identity/financial foundation
2	IR RSS/sitemap + page-aware PDF	Unlock exposure, backlog, catalysts, risk
3	EU/UK/Japan filings	Selected global universe
4	OpenFIGI/security master	Share-class and corporate-action integrity
5	EIA/utility/awards/projects	Independent demand evidence
6	Licensed transcripts/news/market data	Coverage after rights evaluation
7	Postgres + S3 + Temporal	Durable horizontal data plane
8	SSO/RBAC + OTLP + immutable retention	Institutional controls
9	Frozen analyst benchmark	Safe model/provider iteration

SECTION 56

Production readiness acceptance criteria

PLANNED

A production release is ready only when data, model, software, security, operations, and research controls all pass.

Implementation details

- Approved source rights and machine-processing terms.
- Point-in-time global security master for selected universe.
- Parser benchmarks for every enabled content type.
- PostgreSQL migrations and object-store integrity tests.
- Durable orchestration with idempotent activities.
- SSO/RBAC, secret manager, approved egress, signed image, SBOM, vulnerability scans.
- OTLP telemetry, alerts, dashboards, and retention.
- Golden dataset and agent/model release thresholds.
- Backup/PITR and restore/reproduction drill.
- Analyst, methodology, model-risk, data, security, and operations sign-off.

SECTION 57

Operational checklist

IMPLEMENTED

A compact checklist for the person responsible for a real quarterly execution.

Implementation details

- Verify API, database, raw store, and model readiness.
- Check source credentials without displaying values.
- Review last sync failures and freshness.
- Confirm cutoff and selected universe.
- Monitor retries, bytes, injection flags, proposals, and workflow failures.
- Run evaluation and require 100% score reproduction.
- Investigate coverage/confidence and scenario instability.
- Confirm every ranked dossier is approved.
- Publish, hash, archive, compare, and notify.
- Confirm backup and retention jobs captured the new release.

SECTION 58

Volume III references

IMPLEMENTED

Primary implementation and operating specifications used for this volume.

Repository ground truth: `src/aifactory/security.py`, `telemetry.py`, `database.py`, `sources/http.py`, `extraction.py`, `service.py`, `evaluation.py`, `reporting.py`, `api.py`; `tests/*.py`; `pyproject.toml`, `Dockerfile`, `compose.yaml`, `deployment/quarterly-cronjob.yaml`; `docs/evaluation-and-security.md`, `operations.md`, and `deployment/README.md`.

Provider links in Volume I describe external data. This volume focuses on the controls that govern how that data and model output are handled.